

Mayank Kumar

8757836036 | mayankkotlin@gmail.com | [linkedin.com/in/mayank20py](https://www.linkedin.com/in/mayank20py) | github.com/mkkotlin | leetcode.com/u/44_mayank

PYTHON BACKEND ENGINEER | FASTAPI | DJANGO | REST APIs | POSTGRESQL

Experience

Infosys – System Engineer

Oct 2022 – April 2025

Bhubaneswar, India

- Optimized SQL queries by eliminating redundant data retrieval and refining filtering logic, reducing production validation job execution time from approximately 50–70 minutes to 30–45 minutes while preserving business functionality.
- Investigated production incidents by analyzing application logs, SQL query results, and enterprise workflows to identify root causes and support timely issue resolution.
- Developed and enhanced SQL scripts for data validation, troubleshooting, operational reporting, and production support.
- Performed end-to-end analysis of enterprise data flows to identify affected business processes and support production issue resolution.
- Collaborated with cross-functional teams to diagnose application failures, validate fixes, and minimize production downtime.
- Monitored critical production jobs, validated application outputs, and ensured successful execution of business workflows in a large-scale enterprise environment.
- Documented technical findings, troubleshooting procedures, and production resolutions to improve knowledge sharing and future incident handling.

Skills

- **Languages:** Python, TypeScript, SQL
- **System & Infrastructure:** Linux, CI/CD, Networking, Distributed Systems
- **Backend:** FastAPI, Django, Django REST Framework (DRF), SQLAlchemy, JWT Authentication, Django Channels, WebSockets
- **Database:** PostgreSQL, SQLite, IndexedDB
- **Frontend:** React, Angular, RxJS, HTML, CSS, React
- **Tools:** Git, GitHub, Alembic, Postman, Linux, CI/CD
- **Fundamentals:** Object-Oriented Programming (OOP), Data Structures & Algorithms, MVC, Client-Server Architecture, API Design

Projects

Support Ticket System

Tech Stack: Python, FastAPI, SQLAlchemy, PostgreSQL, Alembic, JWT, React, Axios

- Built a full-stack support ticket management system with **FastAPI REST APIs and React**, supporting CUSTOMER, AGENT, and ADMIN roles.
- Implemented **JWT authentication and backend-enforced role-based authorization**, including protected APIs, password hashing, and privileged admin/agent operations.
- Developed APIs for **ticket management, pagination, comments, categories, agent assignment, and status lifecycle** from OPEN → IN_PROGRESS → RESOLVED → CLOSED.
- Integrated **PostgreSQL using SQLAlchemy ORM** and managed version-controlled database schema changes with **Alembic migrations**.
- Deployed the React frontend on **Vercel** and the FastAPI backend on a cloud platform.

Live Project: <https://rticketflow.vercel.app/>

REST API endpoints: <https://backend-4617ffc5.fastapicloud.dev/docs>

Source Code: github.com/mkkotlin/Ticketing-React

Real-Time Collaboration Task Board (Kanban System)

Tech Stack: Django, Django REST Framework, Django Channels, Angular, PostgreSQL, SQLite

- Built a multi-user Kanban task management application supporting real-time synchronization across multiple connected clients using Django Channels and WebSockets.
- Designed REST APIs with Django REST Framework using nested serializers for efficient frontend integration.

- Implemented drag-and-drop task management using Angular CDK with cross-column movement.
- Solved task ordering challenges using bulk update strategies to maintain data consistency.
- Secured APIs with JWT authentication and implemented automatic token refresh.
- Implemented route protection and authentication workflows for secure user access.
- Optimized database access using `select_related` and `prefetch_related` to reduce redundant database queries.

Source Code: github.com/mkkotlin/AsynColl

Offline-First Notes Application

Tech Stack: Django, Django REST Framework, Angular, Dexie.js, IndexedDB

- Built an offline-first note-taking application using IndexedDB (Dexie.js) for persistent local storage.
- Implemented automatic synchronization of locally stored changes after network restoration.
- Developed REST APIs for note synchronization and centralized data management.
- Designed offline/online status indicators and pending synchronization tracking.
- Implemented conflict-aware update handling for improved reliability.

Source Code: github.com/mkkotlin/offline-first-notes-app

NodeLabCanvas (Distributed System Design Playground)

Tech Stack: Django, Django Channels, Angular, RxJS, Tailwind CSS, SVG

- Developed an interactive drag-and-drop distributed system design playground enabling users to model architectures using components such as Clients, API Servers, Databases, Message Queues, Workers, and Load Balancers.
- Implemented dynamic SVG-based connections and node positioning, with automatic persistence of architecture changes using Django backend services.
- Developed a Python-based simulation engine with Django Channels and WebSockets to maintain simulation state and stream real-time updates.
- Developed request-flow and queue visualizations with animated SVG rendering, optimized using Angular change detection, RxJS, and efficient WebSocket event handling.
- Currently extending the simulation engine to support routing strategies, scheduling, and additional distributed system components.

Source Code: github.com/mkkotlin/NodeLabCanvas

ConnectNest | AI-Powered Social Networking Platform

Tech Stack: Django, Django REST Framework, PostgreSQL, AJAX, jQuery, JavaScript, Sentence-Transformers

- Developed a full-stack social networking platform featuring JWT authentication, friend requests, dynamic timelines, and notifications.
- Implemented real-time communication using Django Channels, Daphne, and WebSockets to deliver live notifications and feed updates.
- Implemented JWT-based authentication and authorization to secure APIs and user sessions.
- Built an AI-powered semantic search engine using Sentence-Transformers (all-MiniLM-L6-v2) and cosine similarity to enable context-aware content discovery.
- Optimized semantic search performance by precomputing and storing 384-dimensional vector embeddings in PostgreSQL, eliminating repeated model inference during search requests.
- Improved application startup and request latency by implementing lazy loading for transformer models and optimizing search query execution.

Source Code: github.com/mkkotlin/ConnectNest

Education

B. Tech in Computer Science and Engineering – CGPA: 8.39/10

2018-2022

Achievements

- Solved 200+ LeetCode problems covering core Data Structures & Algorithms.
- Built and deployed **full-stack applications using FastAPI, Django, React, Angular, PostgreSQL, REST APIs, and WebSockets.**
- Hands-on experience developing REST APIs, WebSocket-based systems, AI-powered semantic search, offline-first applications, and distributed system simulations.